

1 Введение в глубокое обучение

1.1 Что такое глубокое обучение?

Несколько десятилетий назад, на заре зарождения компьютерных технологий решение задач компьютерами было строго алгоритмическим: после запуска программы следом за выполнением первой, заранее прописанной инструкции, следовало выполнение следующей, также заранее прописанной инструкции. И так до окончания работы программы.

Логика вызова этих инструкций определялась с помощью логических конструкций на основе простейших логических операций `if`, `else`, операторов `and`, `or` и так далее.

Такой подход способен автоматизировать множество задач. Например, таким образом возможно реализовать логику работы манипулятора на конвейерном производстве. Однако, таким образом очень сложно определять в данных какие либо паттерны, закономерности.

Оцифрованных данных в свою очередь, по мере развития технологий, становилось все больше и больше. Если есть аналитики, способные по определенным данным прогнозировать, например, стоимость недвижимости, почему нельзя научить это делать компьютер, используя статистические методы (как, например, линейная регрессия)?

Такие алгоритмы, которые вместо использования жестких, заранее прописанных правил, способны обучаться на имеющихся данных и извлекать из них закономерности (зачастую даже не видные человеку), являются алгоритмами машинного обучения.

Машинное обучение делится на несколько разделов, из них можно выделить два основных: обучение с учителем и обучение без учителя.

В ходе обучения с учителем модель учится на парах данных “вход” (обозначим его x) и “правильный ответ” (обозначим его y). Такие данные называются размеченными, поскольку для определения правильного ответа при составлении набора данных требуется участие эксперта-разметчика (однако, есть и исключения, о которых мы поговорим позднее).

При обучении без учителя модель работает с не размеченными данными и самостоятельно пытается обнаружить в них закономерности. Глубокое обучение, наоборот, по большей части, является подмножеством обучения с учителем. Оно основано на глубоких, то есть многослойных, нейронных сетях. Глубокие нейронные сети позволяют определять сложные высокоуровневые признаки через распознавание на каждом слое более простых низкоуровневых признаков и их комбинаций.

1.2 Базовые архитектуры нейронных сетей

1.2.1 Полносвязные нейронные сети

Наиболее распространенным слоем среди нейронных сетей является полносвязный линейный слой. В этом слое каждый нейрон связан с каждым

нейроном предыдущего слоя. Связи между нейронами, веса, являются обучаемыми параметрами нейронной сети. Каждый нейрон вычисляет взвешенную сумму входов:

$$y_j = \sum_i x_i w_{ij}$$

Запишем выход полносвязного слоя в виде вектора y :

$$y = \begin{pmatrix} y_1 \\ y_2 \\ \dots \\ y_d \end{pmatrix}^T = \begin{pmatrix} \sum_{j=1} x_j w_{1j} \\ \sum_{j=1} x_j w_{2j} \\ \dots \\ \sum_{j=1} x_j w_{dj} \end{pmatrix}^T = xW$$

Рассматривая выражение $y = xW$ можно заметить, что оно описывает гиперплоскость, проходящую через начало координат. Поэтому, при $x = 0$ всегда $y \equiv 0$ при любых параметрах. Соответственно, при входе $x = 0$ результат глубокой сети, состоящей из полносвязных слоёв, всегда будет нулевым. Добавив в выражение вектор $b \neq 0$ обучаемых параметров

$$y = xW + b,$$

мы получим гиперплоскость, способную проходить где угодно в пространстве признаков.

Попробуем построить сеть из двух слоёв f_1 и f_2 , описываемых параметрами $\{W_1, b_1\}$ и $\{W_2, b_2\}$ соответственно:

$$y_1 = xW_1 + b_1, \quad y_2 = y_1W_2 + b_2$$

Подставим y_1 в выражение для y_2 :

$$y_2 = (xW_1 + b_1)W_2 + b_2 = xW_1W_2 + b_1W_2 + b_2 = x\tilde{W} + \tilde{b}$$

Мы получили из двух слоёв один. Получается, это значит, что глубокие нейронные сети невозможны? "Схлопывание" слоёв происходит из-за линейности функции $y = xW + b$. Для линейной функции выполняется выражение

$$f(ax + by) \equiv af(x) + bf(y),$$

а композиция линейных функций — линейная функция. Поэтому, чтобы сделать глубокие сети возможными, добавим в композицию слоёв нелинейную функцию ϕ :

$$y_1 = \phi(f_1(x)), \quad y_2 = \phi(f_2(y_1))$$

После подстановки, выражение не удастся упростить:

$$y_2 = \phi(\phi(xW_1 + b_1)W_2 + b_2)$$

Нелинейную функцию ϕ называют функцией активации. Нелинейность функции активации также позволяет нейронной сети проще аппроксимировать сложные нелинейные преобразования. От функции активации также должна существовать производная, это свойство потребуется нам в дальнейшем.

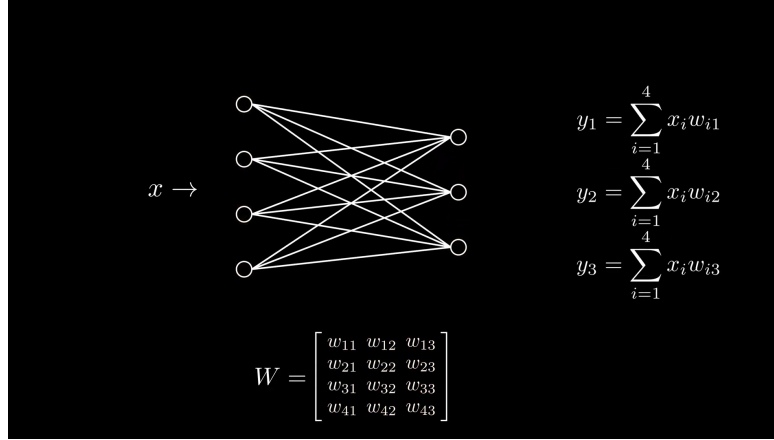


Рис. 1: Схема полносвязного слоя.

Полносвязная нейронная сеть (fully connected network) также называется многослойным перцептроном (multilayer perceptron, MLP) или сетью прямого распространения (feedforward network, FFN).

1.2.2 Функции активации

Исторически, одними из первых широко использовавшихся функций были сигмоида:

$$\sigma(x) = \frac{1}{1 + \exp(-x)}$$

и гиперболический тангенс:

$$\tanh(x) = \frac{\exp(x) - \exp(-x)}{\exp(x) + \exp(-x)}$$

Однако обе функции имеют большой недостаток: при больших по модулю входных значениях они "насыщаются", то есть их значение почти не изменяется при изменении аргумента. Из-за этого их производная будет близка к нулю. О том, почему это плохо, мы поговорим далее.

Сигмоида отражает входное значение в диапазон от 0 до 1, благодаря чему его удобно интерпретировать как вероятность.

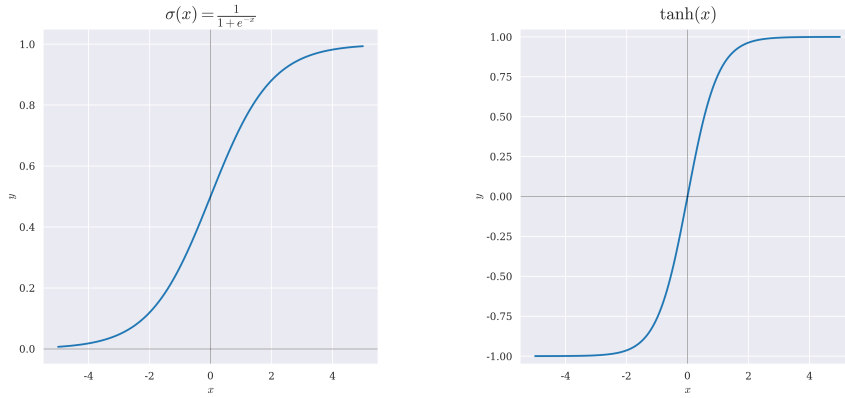


Рис. 2: Графики функций активации сигмоиды (слева) и гиперболического тангенса (справа).

На смену сигмоиде и гиперболическому тангенсу в качестве функции активации пришла функция ReLU (Rectified Linera Unit):

$$\text{ReLU}(x) = \begin{cases} x, & x \geq 0, \\ 0, & x < 0 \end{cases}$$

ReLU нелинейна на $x \in \mathbb{R}$, даже несмотря на линейность лучах $x \geq 0$ и $x < 0$. Ее главное преимущество заключается в простоте вычислений, а также в простоте производной. Но в производной кроется и её главный недостаток: при $x < 0$ производная $\text{ReLU}'(x) \equiv 0$.

Эта проблема решается в модификации ReLU — Leaky ReLU:

$$\text{LeakyReLU}(x) = \begin{cases} x, & x \geq 0, \\ \alpha x, & x < 0, \quad \alpha \neq 0 \end{cases}$$

Обычно, коэффициент α берут небольшим, например $\alpha = 0.1$. Так мы сохраняем нелинейность функции и решаем проблему нулевого градиента.

В PReLU идея развивается дальше и коэффициент α становится обучаемым параметром.

Такие функции как ELU устроены сложнее:

$$\text{ELU}(x) = \begin{cases} x, & x \geq 0, \\ \alpha * (\exp(x) - 1), & x < 0 \end{cases}$$

Их идея заключается в том, чтобы сделать отрицательную часть не линейной, а плавной, чтобы получить более гладкую функцию и, за счёт этого, улучшить динамику обучения (правда, пожертвовав простотой вычислений).

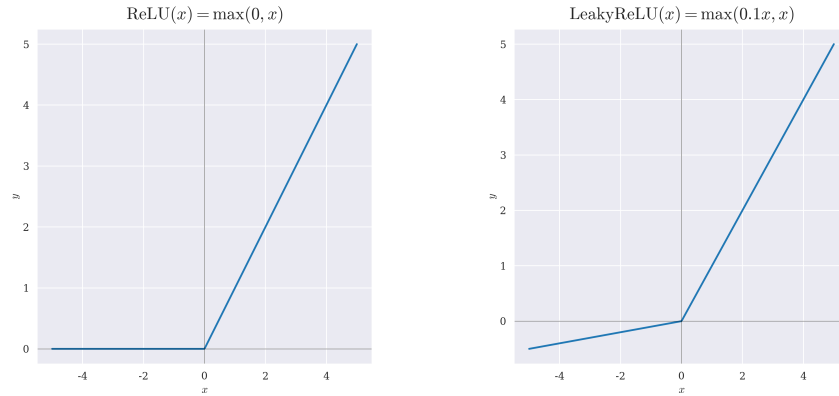


Рис. 3: Графики функций активации ReLU (слева) и LeakyReLU (справа).

Широкое распространение также получила функция SiLU, поскольку является непрерывно гладкой на $\mathbb{R}$:

$$\text{SiLU}(x) = x\sigma(x)$$

И ее обобщение — функция активации SiLU_β :

$$\text{SiLU}_\beta(x) = x\sigma(\beta x)$$

Устремление коэффициента β к бесконечности позволяет превратить SiLU_β в ReLU.

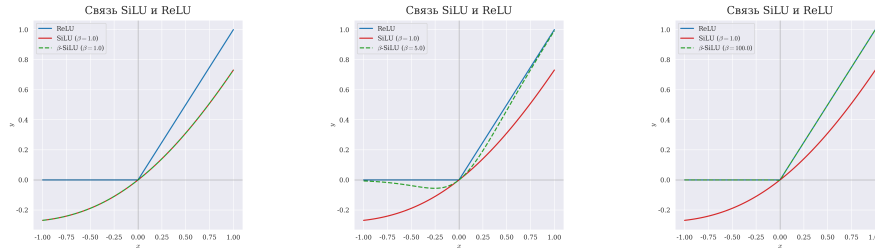


Рис. 4: График функции активации SiLU_β при разных значениях β .

Отдельно стоит рассмотреть функцию Softmax. Формально, она также является функцией активации:

$$\text{Softmax}(x_i) = \frac{\exp(x_i)}{\sum_{j=0}^{\dim(x)} \exp(x_j)}$$

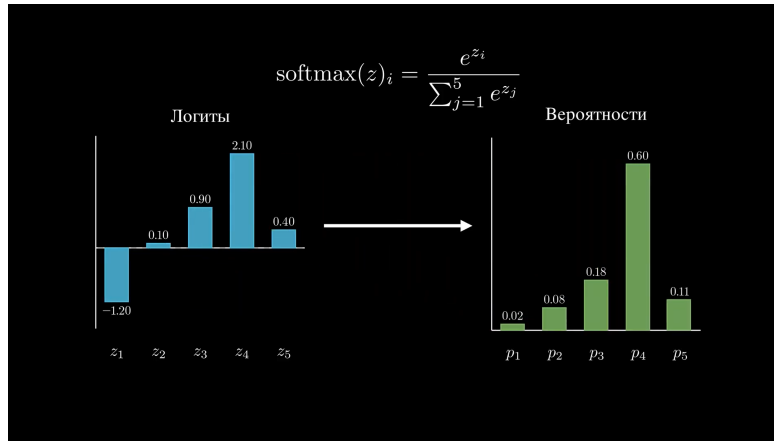


Рис. 5: Преобразование распределения логитов в распределение вероятностей с помощью Softmax.

Её главная особенность заключается в том, что сумма всех компонент вектора, пропущенного через Softmax равна 1, что позволяет трактовать этот вектор как многомерное распределение вероятностей.

1.2.3 Свёрточные нейронные сети

Полносвязные нейронные сети не учитывают пространственных связей в данных. Например в таких, как изображения или текст. Во время обработки, изображение выпрямляется в вектор, после чего каждая компонента обрабатывается как отдельный признак. При этом, если перемешать пиксели изображения, а также соответствующим образом перемешать веса нейронной сети, результат обработки не изменится.

Для обработки таких данных, в которых важна информация о пространственном взаиморасположении признаков, используются свёрточные нейронные сети (convolutional neural network, CNN). Такие сети состоят из свёрточных слоёв. Эти слои применяют к данным фильтры - однослойные полносвязные сети, преобразующие участок данных в число.

В одномерном, простейшем, случае при обработке последовательности данных фильтром размера 3, на первом шаге обрабатываются первые 3 элемента, после чего фильтр смещается на некоторый шаг свёртки (stride) дальше и обрабатывает еще 3 элемента последовательности. Когда последовательность завершается, из результатов обработки формируется карта признаков — новая последовательность, которую также можно обработать свёрточным слоём.

При обработке двумерного одноканального изображения обрабатывается участок размером 3×3 . В этом случае сеть-фильтр имеет 9 параметров (10 в случае наличия смещения). Когда фильтр доходит до края изображения, он опускается вниз на шаг свёртки и смещается к левому краю.

По завершению обработки двумерных данных карта активации получается двумерной.

При обработке многоканального изображения, каждый канал сначала обрабатывается по отдельности, с использованием своего фильтра, а затем полученные карты активаций складываются. Для получения ещё одной карты признаков исходные каналы обрабатываются независимо с использованием новых фильтров.

Важной особенностью свёрточных слоёв является то, что они применяют одинаковый набор параметров к каждому участку изображения. Таким образом, если на изображении присутствует полезный признак (например, наличие вертикальных или горизонтальных линий), то неважно, в какой части он находится, он все равно будет обработан одинаково. Это свойство называется инвариантностью сдвига.

Для свёрточных сетей, также как и для полносвязных, важно использование функции активации.

Отдельно стоит упомянуть выбор размера фильтра. Оказывается, что при использовании соответствующего размера паддинга (заполнения нулями краёв изображения), последовательная обработка изображения двумя свёрточными слоями с фильтрами 3×3 может быть сведена к обработке изображения свёрточным слоем с фильтром 5×5 и наоборот. Аналогично с тремя слоями с фильтрами 3×3 и слоем с фильтром 7×7 . Однако, в первом случае возможно использование дополнительно двух функций активации и всего 20 параметров, а во втором — только одной функции активации при 26 параметрах. Таким образом очевидно, что выгоднее использовать два слоя с фильтрами 3×3 , чем один слой с фильтрами 5×5 , поскольку большее число функций активации способствует большей выразительности нейронной сети. При этом два таких слоя выгоднее по количеству параметров сети.

1.2.4 Рекуррентные нейронные сети

Рекуррентные нейронные сети (RNN) предназначены для работы с последовательными данными, где присутствует зависимость между текущим входом и предыдущими. Такими данными могут быть, например, текст или временные ряды.

$$\begin{cases} h_t = \phi(x_t W + h_{t-1} U + b_1), \\ y_t = \varphi(h_t V + b_2) \end{cases}$$

Главная особенность RNN - наличие обратных связей: сеть способна передать информацию из предыдущих элементов последовательности. Это можно представить в следующем виде:

W , U , V - матрицы параметров, ϕ , φ - активации, b_1 , b_2 - смещения.

h_t - в некотором смысле “память” о предыдущих элементах последовательности. Однако “память” RNN очень короткая и не способна запоминать долгосрочные зависимости. С этой задачей лучше справляется другая рекуррентная архитектура - LSTM.

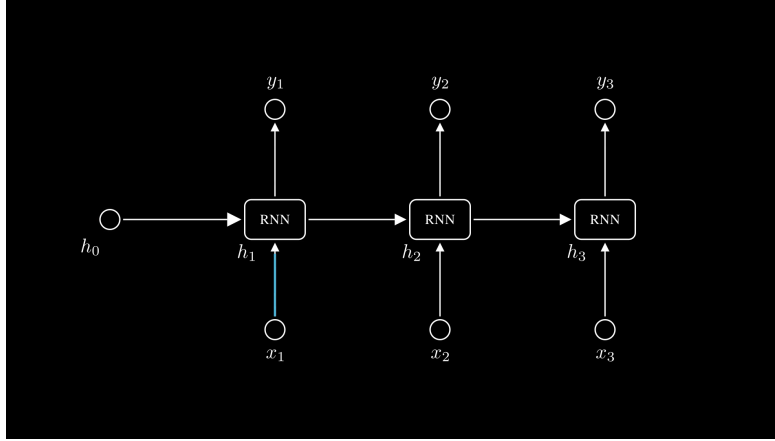


Рис. 6: Схема RNN слоя.

1.3 Обучение нейронных сетей

Нейронные сети обучаются, изменяя свои параметры таким образом, чтобы минимизировать функцию потерь L . Задача обучения становится оптимизационной задачей, и для ее решения будем использовать инструменты из математической оптимизации, а конкретно градиентные оптимизаторы.

Процесс обучения можно разделить на следующие этапы: прямое распространение (forward pass), вычисление **функции потерь** (loss-функции) и обратное распространение (backpropagation) для обновления весов. Данный алгоритм обучения называется алгоритмом **обратного распространения ошибки**.

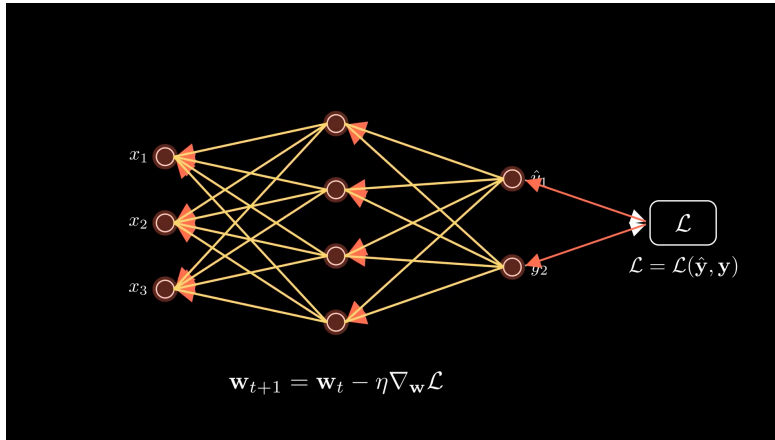


Рис. 7: Обратное распространение ошибки.

1.3.1 Функции потерь

Функция потерь — это оценка стоимости ошибки модели. Пусть y — истинные значения, а $\hat{y}$ — предсказания модели. Простейший способ оценить ошибку — это взять модуль разности по всем объектам и просуммировать их:

$$\text{Absolute Error}(y, \hat{y}) = \sum_{i=1}^N |y_i - \hat{y}_i|$$

Поскольку перед нами стоит задача оптимизации, мы можем применять монотонные возрастающие преобразования и точки минимумов и максимумов не изменятся. Поэтому, разделим абсолютную ошибку на количество объектов, по которым считали ошибку, и получим среднюю абсолютную ошибку (Mean Absolute Error, MAE):

$$\text{MAE}(y, \hat{y}) = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

Также используется средне квадратичная ошибка в качестве функции потерь (Mean Squared Error, MSE). Эта функция сильнее штрафует большие ошибки и слабее — малые.

$$\text{MSE}(y, \hat{y}) = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

Какую же функцию потерь выбрать? Пусть данные устроены следующим образом:

$$y = f(x) + \varepsilon,$$

где ε - случайная величина, шум.

Предположим, что шум центрирован и распределен нормально:

$$\varepsilon \sim \mathcal{N}(0, \sigma^2)$$

Если мы зафиксируем значение x , то значение y тоже будет распределено нормально:

$$y|x \sim \mathcal{N}(f(x), \sigma^2)$$

Запишем плотность такого распределения:

$$p(y|x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp \left[-\frac{1}{2} \left(\frac{y - f_w(x)}{\sigma} \right)^2 \right]$$

Теперь нам нужно понять, какая функция будет лучше соответствовать модели шума ε . Для этого воспользуемся методом максимального правдоподобия.

Необходимо максимизировать величину, называемую правдоподобием:

$$l = \prod_{i=1}^N p(y_i|x_i)$$

Правдоподобие показывает, насколько хорошо выбранные параметры объясняют наблюдаемые данные. В случае, если ответы y - дискретная величина, мы можем говорить, что правдоподобие равно произведению вероятностей выбора правильного ответа моделью.

Произведение оптимизировать неудобно, поэтому вместо него рассматривают логарифм правдоподобия, чтобы превратить произведение в сумму. Также, обычно рассматривается не задача максимизации $\log(l)$, а задача минимизации $-\log(l)$:

$$\text{NLL} = -\sum_{i=1}^N \log p(y_i|x_i) = -\log \left(\frac{1}{\sqrt{2\pi\sigma^2}} \exp \left[-\frac{1}{2} \left(\frac{y - f_w(x)}{\sigma} \right)^2 \right] \right)$$

Упростим:

$$\text{NLL} = \frac{1}{2} \log(2\pi\sigma^2) + \frac{(y - f(x))^2}{2\sigma^2}$$

Здесь $\frac{1}{2} \log(2\pi\sigma^2)$ - это константа, поэтому от нее можно избавиться.

Остаток домножим на $\frac{2\sigma^2}{N}$ (потому что можем это сделать) и получим эквивалентную задачу оптимизации

$$w^* = \operatorname{argmin}_w (\text{NLL}) = \operatorname{argmin}_w \left(\frac{1}{N} \sum_{i=1}^N (y - \hat{y})^2 \right)$$

Таким образом, если шум в данных распределен нормально, то наиболее оптимальная функция потерь — это MSE. Если же шум в данных имеет распределение Лапласа, то наиболее оптимальной функцией потерь будет MAE.

В задаче регрессии шум чаще всего распределен нормально. Шум, распределенный Лапласово, встречается, также может встретиться в задаче регрессии, но чаще его можно обнаружить, например, в задаче генерации изображений.

А вот для задачи классификации не подойдут ни MAE, ни MSE, поскольку шум в такой задаче имеет категориальное распределение. Чтобы определить оптимальную функцию потерь, воспользуемся теорией информации.

Пусть имеется C классов. Тогда для объекта x модель f выдает вектор вероятностей

$$q(x) = (q_1(x), q_2(x), \dots, q_C(x))$$

Если некоторое событие X произошло с вероятностью p , то количество информации, которое несёт это событие (мера самоинформации), равно

$$I[X] = -\log p$$

отсюда видно, что маловероятное событие несет больше информации, чем вероятное. Математическое ожидание информации в одном наблюдении называется энтропией (Шенноновской энтропией):

$$H(p) = \mathbb{E}[I[X]] = -\sum_{i=1}^C p_i \log p_i$$

Теперь предположим, что истинное распределение классов — p , а модель предсказывает распределение q . Тогда среднее количество информации, которое потребуется для описания истинных ответов с использованием распределения q , задаётся кросс-энтропией:

$$H(p, q) = -\sum_{i=1}^C p_i \log q_i$$

В задаче обучения с учителем p - это вектор, у которого $p_c = 1$ для правильного класса, а для остальных $p_i = 0$. Тогда можем упростить:

$$H(p, q) = -\log q_c,$$

где q_c - вероятность, которую модель присвоила правильному классу.

Если мы решим применить метод максимального правдоподобия для решения задачи классификации, то для одного объекта получим:

$$\text{NLL} = -\log q_c = H(p, q)$$

Значит мы можем записать функцию потерь кросс-энтропии:

$$L_{CE} = \frac{1}{N} \sum_{n=1}^N H(p^{(n)}, q^{(n)}) = -\frac{1}{N} \sum_{n=1}^N \log q_{c^{(n)}}^{(n)}$$

Но что эта функция значит? Какая интуиция? Вернемся к полной формуле кросс-энтропии и немного перепишем её:

$$H(p, q) = -\sum_i p_i \log q_i = -\underbrace{\sum_i p_i \log p_i}_{H(p)} + \underbrace{\sum_i p_i \log \frac{p_i}{q_i}}_{D_{KL}(p||q)},$$

Таким образом:

$$H(p, q) = H(p) + D_{KL}(p||q),$$

где:

- $D_{KL}\langle p||q\rangle$, — расхождение Кульбака-Лейблера, "расстояние" от распределения p к распределению q (на самом деле это не расстояние, поскольку расхождение Кульбака-Лейблера не обладает свойством симметричности).
- $H(p)$ — Шеннонова энтропия данных, мера "шума". Это — теоретический минимум для корректной функции потерь.

Таким образом, минимизация кросс-энтропии означает минимизацию расхождения Кульбака-Лейблера, а значит приближает моделируемое распределение q к истинному распределению p .

1.3.2 Обратное распространение ошибки

Пусть f - функция, которая представляет нашу нейронную сеть, а $f_1, f_2, \dots, f_n$ — ее слои. Этап прямого распространения тогда записывается просто:

$$y = f(x) = f_1 \circ f_2 \circ \dots \circ f_n(x)$$

После прямого распространения и вычисления функции потерь, необходимо вычислить градиент $\nabla L(y, \hat{y})$. После чего используется алгоритм градиентного спуска для обновления параметров:

$$w_t = w_{t-1} - \lambda \nabla L_{w_{t-1}}(y, \hat{y})$$

Этап обратного распространения заключается в вычислении градиента ∇L_w . Градиент — это вектор частных производных:

$$\nabla L_w = \left(\frac{\partial L}{\partial w_1} \quad \frac{\partial L}{\partial w_2} \quad \dots \quad \frac{\partial L}{\partial w_N} \right)$$

Переменные $w_1, w_2, \dots$ — это параметры нейронной сети, а именно веса конкретных слоёв. Допустим, $w_1 = (W_1)_{ij}$ - это параметр слоя f_1 , а $w_2 = (W_2)_{ij}$ - параметр слоя f_2 .

Найдем $\frac{\partial L}{\partial w_1}$. Для этого продифференцируем сложную функцию с помощью цепного правила:

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial y} \frac{\partial y}{\partial w_1}$$

$\frac{\partial L}{\partial y}$ — это просто производная функции потерь, вычисляется достаточно

легко. Найдем производную выхода слоя по параметру $\frac{\partial y}{\partial w_1}$.

Пусть

$$z_1 = f_2 \circ \dots \circ f_n(x)$$

Тогда:

$$y = f_1 \circ f_2 \circ \dots \circ f_n(x) = f_1(z_1) = z_1 W_1 + b_1$$

Поскольку y — это вектор, то только одна его компонента зависит от элемента $w_1 = (W_1)_{ij}$, и это компонента y_j . Вычислим её:

$$y_j = \sum_k z_{1k} (W_1)_{kj} + b_{1j}$$

Теперь найти $\frac{\partial y}{\partial w_1}$ достаточно просто:

$$\frac{\partial y_k}{\partial w_1} = 0 \quad \forall k \neq j,$$

$$\frac{\partial y_j}{\partial w_1} = z_{1i}$$

Значит, векторная производная:

$$\frac{\partial y}{\partial w_1} = z_{1i} e_j,$$

где e_j — это единичный вектор, все компоненты которого, кроме компоненты на j -ой позиции, равны 0.

Теперь аналогичным образом попробуем найти $\frac{\partial L}{\partial w_2}$:

$$\frac{\partial L}{\partial w_2} = \frac{\partial L}{\partial z_1} \frac{\partial z_1}{\partial w_2}$$

Разберем это выражение по порядку. Производная $\frac{\partial z_1}{\partial w_2}$ нам знакома — это производная выхода второго слоя по параметру, мы только что вычисляли аналогичную производную для первого слоя, дальнейшие математические выкладки остаются внимательному читателю в качестве самостоятельной работы.

А производная $\frac{\partial L}{\partial z_1}$ нам не знакома. Снова воспользуемся цепным правилом:

$$\frac{\partial L}{\partial z_1} = \frac{\partial L}{\partial y} \frac{\partial y}{\partial z_1}$$

Производную $\frac{\partial L}{\partial y}$ мы уже считали. А вот $\frac{\partial y}{\partial z_1}$ — это производная выхода слоя f_1 по его входу (а вход f_1 слоя — это выход f_2 слоя). Таким образом становится понятно, что:

1. На каждом слое, исключая f_n , нам необходимо считать 2 производные $\frac{\partial L}{\partial w}$ и $\frac{\partial L}{\partial z}$ — производную по параметру, которая нам нужна для вычисления градиента, и производную по входу, чтобы с помощью нее вычислить производные следующего слоя.

2. Невозможно вычислить производные слоя f_k , не вычислив производные слоев $f_1, f_2, \dots, f_{k-1}$. Это ограничение задает строгое направление обратного распространения ошибки.

3. Для вычисления $\frac{\partial L}{\partial w_k}$, производной на k -ом слое, нам необходимо найти произведение $\frac{\partial L}{\partial y} \frac{\partial y}{\partial z_1} \frac{\partial z_1}{\partial z_2} \frac{\partial z_2}{\partial z_3} \dots \frac{\partial z_{k-2}}{\partial z_{k-1}}$.

Если каждая производная $\frac{\partial y}{\partial z}$ этого произведения будет меньше 1 по модулю, например $\frac{\partial y}{\partial z} = 0.5$, то для седьмого слоя мы получим множитель $(0.5)^6 \frac{\partial L}{\partial y} \approx 0.016 \frac{\partial L}{\partial y}$.

Эта проблема называется проблемой угасающих градиентов (или исчезающих градиентов). В некоторых случаях возможна противоположная ситуация — градиенты "взрываются". Из-за этого мы (пока) не можем обучать по настоящему глубокие сети.

Для вычисления на компьютере алгоритма обратного распространения ошибки строится граф вычислений. Узлы этого графа — простейшие операции, такие как сложение, умножение, деление и тд, а ребра отображают их последовательность. Мы можем создавать собственные операции, объединяя однотипные элементы в один узел, тем самым упрощая граф. Во время прямого распространения выполнение операций осуществляется последовательно, а при вычислении градиента — в обратном порядке.

1.3.3 Градиентный спуск

Вычислив градиент ∇L_w , мы можем обновить параметры нейронной сети с помощью градиентного спуска.

Стандартный алгоритм градиентного спуска вычисляется по всей выборке:

$$w_{t+1} = w_t - \lambda \nabla L_{w_t},$$

$$\nabla L_{w_t} = \sum_{i=1}^N \nabla l_{w_t}(y_i, \hat{y}_i)$$

где N - количество объектов выборки.

Очевидный недостаток такого подхода — потребность в огромном количестве памяти и вычислительных ресурсов. Не самый очевидный недостаток — высока вероятность попадания в локальные минимумы или седловые точки функции ошибки.

Стохастический градиентный спуск

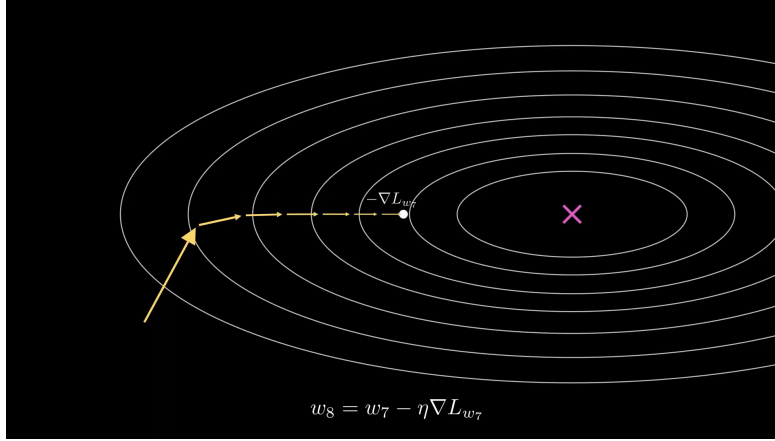


Рис. 8: Траектория градиентного спуска.

Решение — реализуем градиентный спуск не по всему набору данных, а только по его части — по мини-батчу. Объекты в мини-батче выбираются случайным образом, делая процесс стохастическим. Такой оптимизатор называется стохастическим градиентным спуском (SGD).

$$w_{t+1} = w_t - \lambda \nabla L_{w_t},$$

$$\nabla L_{w_t} = \sum_{i=1}^B \nabla l_{w_t}(y_i, \hat{y}_i)$$

где B - количество объектов минибатча.

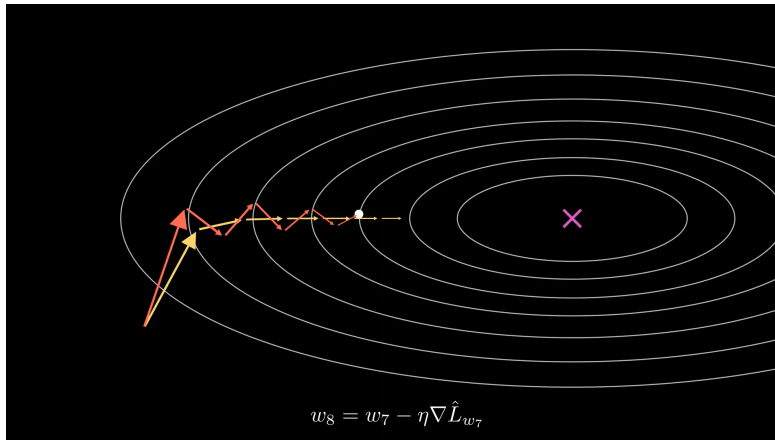


Рис. 9: Траектория стохастического градиентного спуска.

Реальные данные чаще всего шумные, а значит и градиент при вычислении SGD также будет шумным. Чем больше размер батча, тем точнее градиент. Но также этот шум позволяет избегать седловых точек и некоторых локальных минимумов.

Momentum

Градиентный спуск можно представить как шарик, скатывающийся по поверхности функции. Шарик, будучи объектом физическим, обладает скоростью. Реализация "скорости" в SGD (через добавление изменения весов за предыдущий шаг) называется momentum:

$$w_t = w_{t-1} - \lambda \nabla L_{w_{t-1}} + \alpha(w_{t-1} - w_{t-2})$$

Это выражение можно записать иначе:

$$\begin{cases} v_t = \alpha v_{t-1} - \lambda \nabla L_{w_{t-1}}, \\ w_t = w_{t-1} + v_t \end{cases}$$

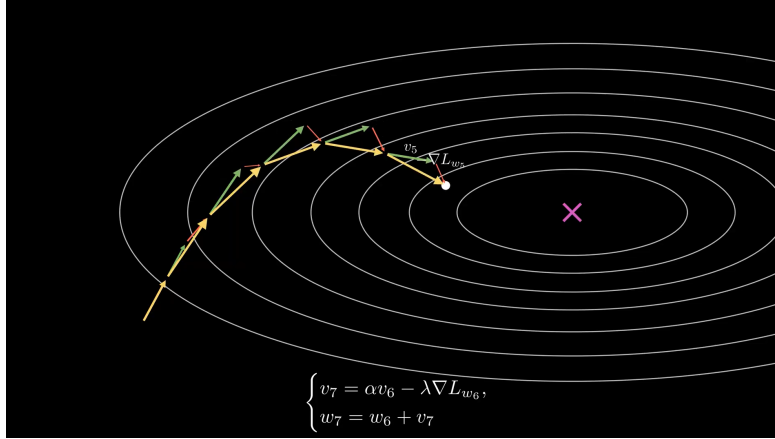


Рис. 10: Траектория momentum.

Nesterov momentum Для momentum существует модификация — Nesterov momentum. Его идея заключается в том, чтобы считать градиент не в текущей точке, а в точке, в которую попали бы, следуя импульсу:

$$\begin{cases} v_t = \alpha v_{t-1} - \lambda \nabla L_{w_{t-1} + \alpha v_{t-1}}, \\ w_t = w_{t-1} + v_t \end{cases}$$

Данный момент как бы “заглядывает в будущее”, корректируя ошибку оптимизации на текущем шаге.

Скорость обучения

Вычисление градиента — дорогая задача, поэтому хотелось бы найти оптимальные параметры нейронной сети за как можно меньшее число шагов

градиентного спуска. Конечно, momentum и nesterov momentum ускоряют обучение сети, однако на скорость обучения больше всего влияет гиперпараметр λ . Чем больше λ , тем быстрее обучается модель. Однако в конце обучения хотелось бы, чтобы шаг был как можно меньше, чтобы добиться большей точности.

Пусть λ — это некоторая функция $\lambda(t)$. Тогда возможно просто подобрать подходящую функцию, чтобы значение λ соответствовало нашим желаниям. Такая техника называется **learning rate schedule**.

Стоит отметить, что параметры модели инициализируются случайным образом и градиенты на первых шагах могут быть слишком шумными. Тогда, большой шаг обучения может отбросить параметры в “плохую” часть функции потерь. Поэтому, часто скорость обучения запускают с малых, околонулевых, значений, постепенно, разогревая до “пика”, после чего она начинает затухать. Такая техника называется **warm-up**.

Adagrad

Learning rate schedule не идеален, поскольку основан на предположении, что заранее известно, на каких участках шаг обучения должен быть большим, а на каких — малым. Поэтому существуют методы адаптивной настройки скорости обучения. Первый такой метод — **Adagrad**:

$$\begin{cases} r_t = r_{t-1} + \left(\nabla L_{w_{t-1}} \right)^2, \\ w_t = w_{t-1} - \frac{\lambda}{\sqrt{r_t + \epsilon}} \nabla L_{w_{t-1}} \end{cases}$$

Этот метод старается компенсировать слишком большие и слишком малые компоненты градиента.

RMSProp

Модификация Adagrad — RMSProp (root mean square propagation). Идея этого оптимизатора заключается в том, чтобы не складывать градиенты, а брать скользящее среднее:

$$\begin{cases} r_t = \beta r_{t-1} + (1 - \beta) \left(\nabla L_{w_{t-1}} \right)^2, \\ w_t = w_{t-1} - \frac{\lambda}{\sqrt{r_t + \epsilon}} \nabla L_{w_{t-1}} \end{cases}$$

Adam

Дело осталось за малым — объединить RMSProp и momentum и получить Adam (Adaptive momentum):

$$\begin{cases} v_t = \beta_1 v_{t-1} + (1 - \beta_1) \nabla L_{w_{t-1}}, \\ r_t = \beta_2 r_{t-1} + (1 - \beta_2) \left(\nabla L_{w_{t-1}} \right)^2 \end{cases}$$

Поскольку v_0 и r_0 инициализируются нулями, то в их оценке есть смещение. Для их коррекции в Adam присутствует следующий шаг:

$$\begin{cases} \hat{v}_t = \frac{v_t}{1 - \beta_1}, \\ \hat{r}_t = \frac{r_t}{1 - \beta_2} \end{cases}$$

Следом идёт само обновление весов:

$$w_t = w_{t-1} - \lambda \frac{\hat{v}_t}{\sqrt{\hat{r}_t + \epsilon}}$$

Adam является серебряной пулей при оптимизации параметров нейронной сети.

1.3.4 Регуляризация

Нейронные сети обучаются на некотором тренировочном наборе данных, однако работать им приходится с теми данными, которые они, скорее всего, не видели (тестовые данные). При вычислении функции потерь на разных наборах данных возможны следующие сценарии: высокая ошибка как на тренировочном, так и на тестовом наборах данных, либо высокая ошибка на тестовых данных, но на тренировочных — почти нулевая. Говорят, что в первом случае модель недообучилась, а во втором — переобучилась.

Пусть данные генерируются с помощью следующего генератора:

$$y = f(x) + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \sigma^2)$$

Поскольку реальные данные шумные, то всегда будет присутствовать шум, который невозможно предсказать на тестовых данных. Высокая тестовая ошибка может быть результатом недостаточной “подгонки” модели, если она слишком простая (или мало обучалась) и не способна улавливать связи в данных. Также, тестовая ошибка может быть обеспечена слишком точным следованием за шумом на тренировочных данных, из-за чего итоговая функция не способна дать правильное предсказание на тестовых данных.

Среднеквадратичная ошибка может быть разложена на три составляющие:

$$\text{expected loss} = (\text{bias})^2 + \text{variance} + \text{noise}$$

Bias (смещение) — это систематическая ошибка модели, а **variance** (разброс) — оценка чувствительности модели к изменчивости данных. Недообучение выражается в высоком смещении и низком разбросе. Переобучение — в высоком разбросе и низком смещении. Хорошо обученная модель — компромисс между смещением и разбросом (**bias-variance tradeoff**).

Чтобы уменьшить bias, нужно увеличить количество параметров модели.

Уменьшить variance возможно через уменьшение сложности модели. В этом помогают методы регуляризации. При этом будет увеличиваться bias.

Регуляризация позволяет достичь компромисса для более сложной модели на тех же данных. При обучении нейронных сетей чаще всего используются l_1 и l_2 регуляризации (в общем случае - **weight decay**), идея которых заключается в добавлении нормы весов как дополнительного компонента функции потерь:

$$L_{reg_1} = L + c\|w\|_1,$$

$$L_{reg_2} = L + c\|w\|_2^2,$$

Weight decay “сдерживает” веса нейронной сети у нуля, уменьшая тем самым сложность модели. Сложность модели уменьшает и другой метод регуляризации, называемый **dropout**. Основная идея dropout — зануление некоторых выходов слоя случайным образом.

Чтобы понизить variance, не уменьшая сложность модели, необходимо расширить набор данных. Если добавить новые объекты не представляется возможным, можно воспользоваться **аугментацией данных**.

1.3.5 Нормализация

Данные имеют разный масштаб. Некоторые признаки могут быть близки к нулю, некоторые — колебаться вблизи некоторого числа. Большие по модулю значения модель будет считать важнее, даже если это не так. Для избежания этого данные нормализуются:

$$\tilde{x}^i = \frac{x^i - \mu^i}{\sigma^i + \epsilon}, \quad \mu^i = \frac{1}{N} \sum_{n=1}^N x_n^i, \quad \sigma^i = \sqrt{\frac{1}{N} \sum_{n=1}^N (x_n^i - \mu^i)^2}$$

Однако, распределение входов каждого слоя модели (кроме первого) также будет изменяться на каждом шаге обучения, поскольку изменяются параметры модели. Это означает, что каждому слою приходится заново переучиваться на новое распределение. Для решения этой проблемы применяется нормализация по батчу:

$$\hat{z} = \frac{z - \mu_B}{\sigma_B + \epsilon}$$

Но нормализуя данные таким образом, мы уменьшаем предсказательную способность сети. Поэтому результат ремасштабируется с помощью обучаемых параметров γ и β :

$$\tilde{z} = \gamma \hat{z} + \beta$$

Позже, интерпретацию с изменением распределений входов слоев стали считать недостаточной. Было выяснено, что эффект успеха нормализации заключается в том, что поверхность функции потерь становится более гладкой, а градиенты — более предсказуемыми.